

TP 3: Metrics d'évaluation des modeles

October 8, 2023

1 Introductions

L'évaluation des modèles est une étape essentielle dans le domaine de l'apprentissage automatique. Elle nous permet de mesurer la performance de nos modèles et de prendre des décisions éclairées sur leur déploiement ou leur amélioration. Dans ce TP, nous explorerons en profondeur les techniques d'évaluation de modèles, en mettant l'accent sur des aspects tels que les métriques d'évaluation, la validation croisée, les courbes d'apprentissage, les courbes de validation, et la recherche de grille (GridSearch CV).

Objectifs du TP :

- Compréhension des Métriques d'Évaluation : Notre premier objectif sera de comprendre en détail les métriques d'évaluation les plus couramment utilisées dans l'apprentissage automatique. Nous explorerons des métriques telles que l'exactitude (accuracy), le F1-score, la précision (precision), le rappel (recall), et bien d'autres. Vous apprendrez à choisir la métrique appropriée en fonction de votre problème spécifique.
- Validation Croisée : Nous étudierons la validation croisée (cross-validation) comme une technique essentielle pour évaluer la performance d'un modèle de manière robuste. Vous apprendrez à mettre en œuvre la validation croisée avec différentes méthodes, telles que la validation croisée en k-fold et la validation croisée stratifiée.
- Courbes d'Apprentissage et Courbes de Validation : Les courbes d'apprentissage et de validation sont des outils puissants pour diagnostiquer la performance d'un modèle, identifier les problèmes de surajustement (overfitting) ou de sous-ajustement (underfitting), et estimer la taille optimale de l'ensemble de données. Nous examinerons comment les construire et les interpréter.
- Recherche de Grille (GridSearch CV) : La recherche de grille est une technique permettant de sélectionner les meilleurs hyperparamètres pour un modèle donné. Nous verrons comment utiliser GridSearch CV pour optimiser nos modèles.
- Objectif de Modélisation : Enfin, nous comprendrons comment ces techniques d'évaluation peuvent nous aider à définir des objectifs de modélisation clairs et à choisir les modèles les plus appropriés pour nos problèmes.

À la fin de ce TP, vous disposerez des compétences essentielles en évaluation de modèles nécessaires pour mesurer la performance de vos algorithmes d'apprentissage automatique. Vous serez capable de choisir les métriques appropriées, de comprendre les avantages de la validation croisée, d'analyser les courbes d'apprentissage et de validation, et de rechercher les hyperparamètres optimaux pour vos modèles. Préparez-vous à maîtriser ces techniques cruciales pour la création de

modèles d'apprentissage automatique de haute qualité.

```
[32]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import seaborn as sns
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
plt.style.use('ggplot')
```

```
[33]: df = pd.read_excel('Datasets/titanic3.xls')
df = df.drop(['name', 'sibsp', 'parch', 'sex', 'ticket', 'cabin', 'embarked',
↳ 'boat', 'body', 'home.dest'], axis=1)
df = df.dropna(axis=0)

y = df['survived']
X = df.drop('survived', axis=1)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳ random_state=5)
X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

```
[33]: ((836, 3), (209, 3), (836,), (209,))
```

```
[34]: model = KNeighborsClassifier(n_neighbors=8)
model.fit(X_train, y_train)
model.score(X_train, y_train), model.score(X_test, y_test)
```

```
[34]: (0.7344497607655502, 0.5980861244019139)
```

2 Métriques d'Évaluation des Modèles

2.1 Métriques de Classification

Dans cette sous-section, nous allons explorer différentes métriques couramment utilisées pour évaluer les performances des modèles de classification.

La **précision** mesure la proportion des prédictions positives correctes parmi toutes les prédictions positives et négatives. Elle est particulièrement utile lorsque les faux positifs sont coûteux.

La **rappel** mesure la proportion des instances réellement positives prédites correctement parmi toutes les instances réellement positives et négatives. Il est important lorsque les faux négatifs sont coûteux.

Le **F1-score** est une métrique qui combine la précision et le rappel en une seule mesure, utile lorsque vous voulez trouver un équilibre entre la précision et le rappel.

```
[35]: from sklearn.metrics import
      accuracy_score, precision_score, f1_score, recall_score, roc_auc_score, roc_curve, classification
      y_pred = model.predict(X_test)

      print("Accuracy: ", accuracy_score(y_test, y_pred))
      print("Precision: ", precision_score(y_test, y_pred))
      print("F1: ", f1_score(y_test, y_pred))
      print("Recall: ", recall_score(y_test, y_pred))
```

```
Accuracy:  0.5980861244019139
Precision:  0.4626865671641791
F1:  0.4246575342465754
Recall:  0.3924050632911392
```

2.2 Score AUC-ROC

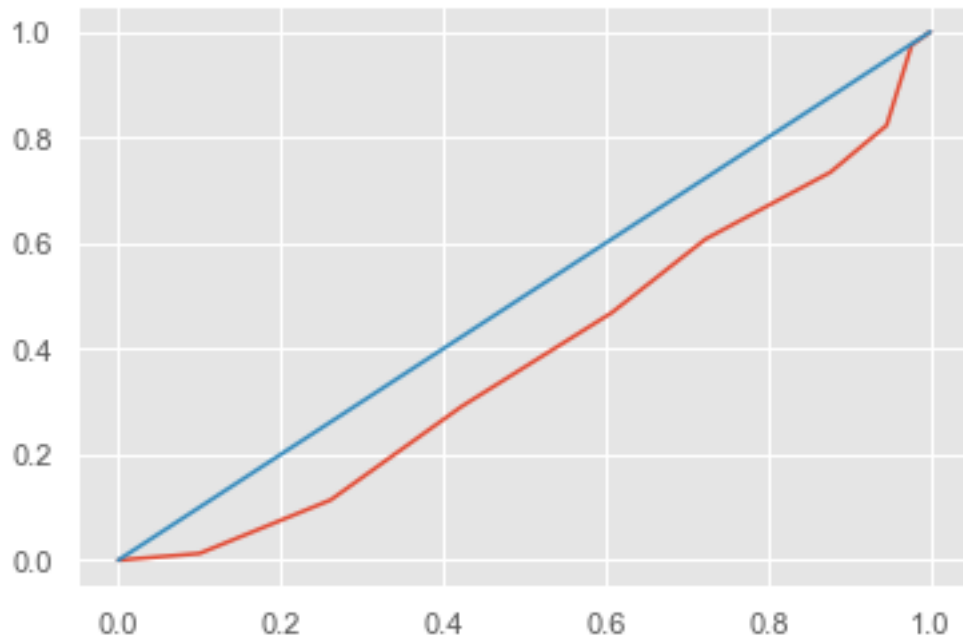
Le **score AUC-ROC** (Aire Sous la Courbe de Caractéristique de Fonctionnement du Récepteur) mesure la capacité d'un modèle à distinguer les classes positives et négatives. Une valeur plus élevée indique de meilleures performances.

```
[36]: y_score= model.predict_proba(X_test)[: ,0]
      print(roc_auc_score(y_test, y_score))
      fpr, tpr, _=roc_curve(y_test, y_score)

      plt.plot(fpr,tpr)
      plt.plot([0,1],[0,1])
```

```
0.38330087633885107
```

```
[36]: [<matplotlib.lines.Line2D at 0x7f8d87017d00>]
```



2.3 Matrice de Confusion

La **matrice de confusion** est une table qui permet de visualiser les performances d'un modèle de classification. Elle affiche le nombre de vrais positifs, de vrais négatifs, de faux positifs et de faux négatifs. C'est un outil essentiel pour comprendre comment un modèle se comporte.

```
[37]: from sklearn.metrics import confusion_matrix
      confusion_matrix(y_test, model.predict(X_test))
```

```
[37]: array([[94, 36],
           [48, 31]])
```

```
[38]: cm = confusion_matrix(y_test, y_pred)

print(classification_report(y_test, y_pred))
print(f'ROC AUC score: {roc_auc_score(y_test, y_score)}')
print('Accuracy Score: ', accuracy_score(y_test, y_pred))

# Visualizing Confusion Matrix
plt.figure(figsize = (8, 5))
sns.heatmap(cm, cmap = 'Blues', annot = True, fmt = 'd', linewidths = 5, cbar =
    ↪ False, annot_kws = {'fontsize': 15},
            yticklabels = ['No survived', 'survived'], xticklabels =
    ↪ ['Predicted no survived', 'Predicted survived'])
plt.xticks(rotation = 0)
```

```

plt.show()

# Roc AUC Curve
false_positive_rate, true_positive_rate, thresholds = roc_curve(y_test, y_score)
roc_auc = roc_auc_score(y_test, y_score)

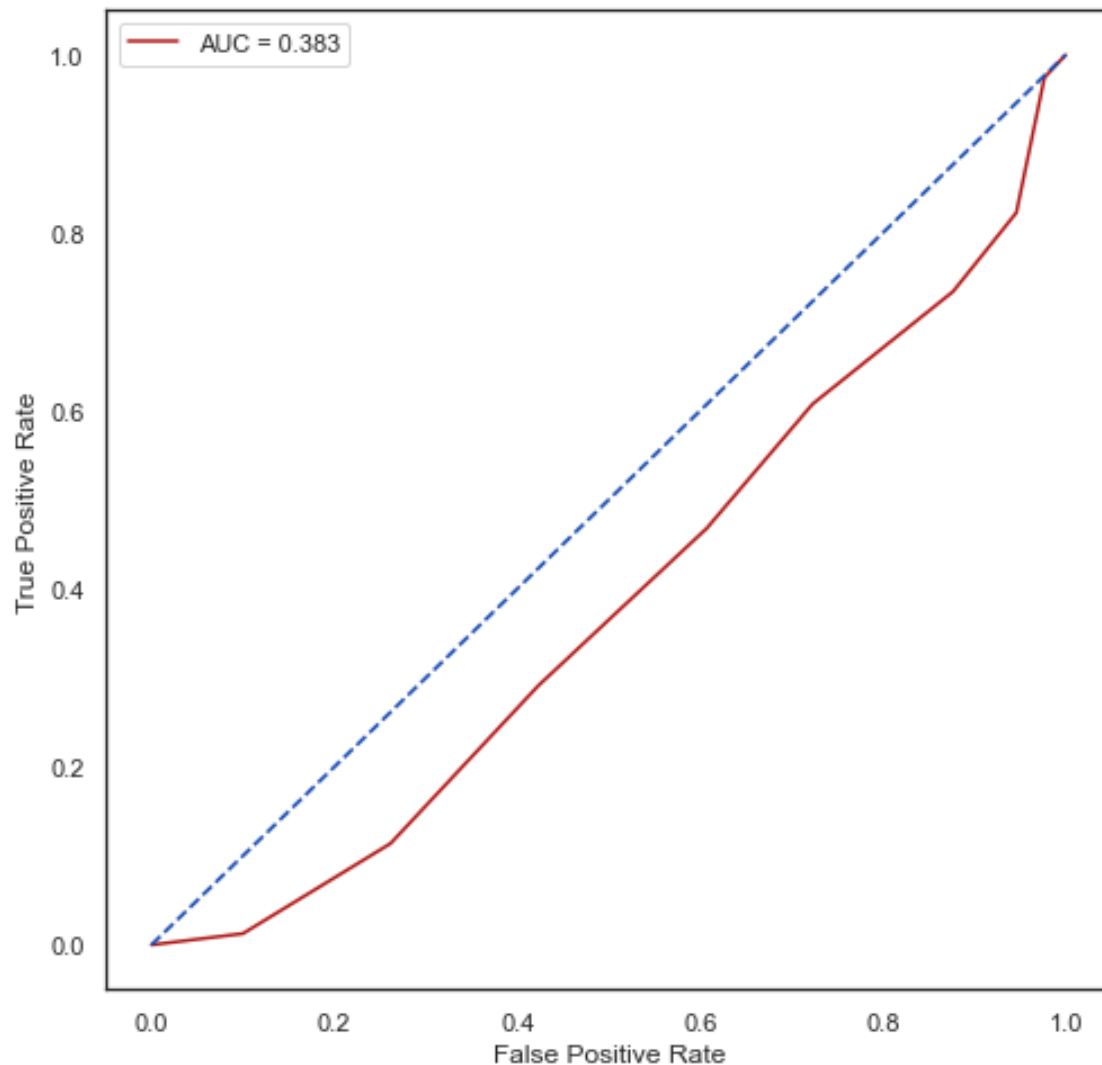
sns.set_theme(style = 'white')
plt.figure(figsize = (8, 8))
plt.plot(false_positive_rate,true_positive_rate, color = '#b01717', label = 'ROC AUC = %0.3f' % roc_auc)
plt.legend(loc = 'lower right')
plt.plot([0, 1], [0, 1], linestyle = '--', color = '#174ab0')
plt.axis('tight')
plt.ylabel('True Positive Rate')
plt.xlabel('False Positive Rate')
plt.legend()
plt.show()

```

	precision	recall	f1-score	support
0	0.66	0.72	0.69	130
1	0.46	0.39	0.42	79
accuracy			0.60	209
macro avg	0.56	0.56	0.56	209
weighted avg	0.59	0.60	0.59	209

ROC AUC score: 0.38330087633885107
Accuracy Score: 0.5980861244019139

No survived	94	36
survived	48	31
	Predicted no survived	Predicted survived



3 Ensemble de validation et validation croisée

Dans cette sous-section, nous explorerons les techniques de validation set qui sont utilisées pour évaluer les performances des modèles.

La **validation set simple** consiste à diviser les données en un ensemble d'entraînement et un ensemble de validation. C'est la méthode la plus basique, mais elle peut être sujette au biais de sélection des données.

La **validation croisée** est une technique plus robuste pour évaluer les performances des modèles en utilisant différentes partitions des données.

```
[39]: from sklearn.model_selection import cross_val_score
```

```
[40]: cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')
```

```
[40]: array([0.61904762, 0.67065868, 0.71856287, 0.67065868, 0.7005988 ])
```

3.1 Kfold

La **validation croisée K-Fold** divise les données en K partitions égales. Le modèle est entraîné K fois, chaque fois en utilisant K-1 partitions pour l'entraînement et la partition restante pour la validation.

```
[41]: from sklearn.model_selection import KFold

cv= KFold()
cross_val_score(model, X_train, y_train, cv=cv, scoring='accuracy')
```

```
[41]: array([0.69047619, 0.64670659, 0.67664671, 0.64670659, 0.68263473])
```

3.2 ShuffleSplit

La **validation croisée Shuffle Split** mélange les données et les divise aléatoirement en ensembles d'entraînement et de validation pour chaque itération.

```
[42]: from sklearn.model_selection import ShuffleSplit

cv= ShuffleSplit(n_splits=5, test_size=.25, random_state=0)
cross_val_score(model, X_train, y_train, cv=cv, scoring='accuracy')
```

```
[42]: array([0.65550239, 0.6076555 , 0.64593301, 0.70334928, 0.64114833])
```

3.3 LeaveOneOut

La **validation croisée Leave-One-Out** utilise chaque échantillon comme ensemble de validation à tour de rôle. Cela signifie que K-Fold Cross Validation avec K égal au nombre total d'échantillons.

```
[ ]: from sklearn.model_selection import LeaveOneOut

cv= LeaveOneOut()
cross_val_score(model, X_train, y_train, cv=cv, scoring='accuracy')
```

3.4 StratifiedKFold

La **validation croisée Stratified K-Fold** est similaire à K-Fold Cross Validation, mais elle garantit que la répartition des classes dans chaque partition est similaire à celle de l'ensemble de données global.

```
[43]: from sklearn.model_selection import StratifiedKFold

cv= StratifiedKFold(n_splits=5, shuffle=True)
```



```
cross_val_score(model, X_train, y_train, cv=cv, scoring='accuracy')
```

```
[43]: array([0.64285714, 0.61676647, 0.64071856, 0.7005988 , 0.68862275])
```

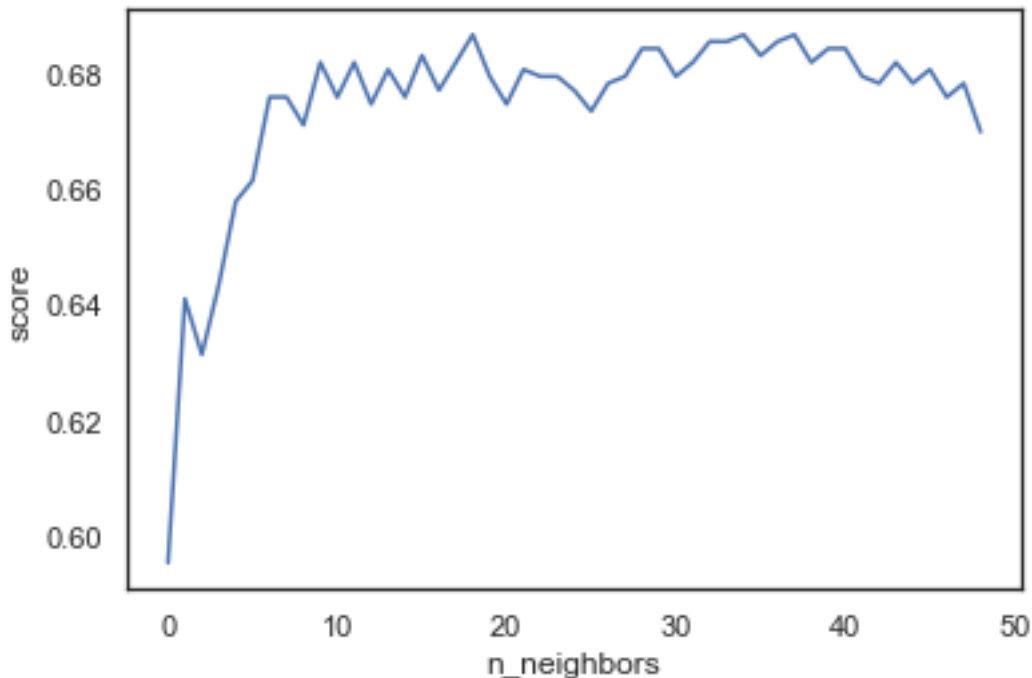
Ce code illustre comment utiliser la validation croisée (cross-validation) pour sélectionner le meilleur hyperparamètre `n_neighbors` pour un modèle K-Nearest Neighbors (KNN).

`score = cross_val_score(KNeighborsClassifier(n_neighbors=k), X_train, y_train, cv=5).mean()`: À chaque itération de la boucle, le code crée un modèle KNN avec `n_neighbors` égal à la valeur actuelle de `k`. Ensuite, il utilise la validation croisée (`cross_val_score`) avec 5 plis (`cv=5`) pour évaluer les performances du modèle. La méthode `.mean()` calcule la moyenne des scores de validation croisée pour ces plis. Le score moyen est ensuite stocké.

```
[44]: val_score = []
      for k in range(1, 50):
          score = cross_val_score(KNeighborsClassifier(n_neighbors=k), X_train,
          ↪ y_train, cv=5).mean()
          val_score.append(score)

      plt.plot(val_score)
      plt.ylabel('score')
      plt.xlabel('n_neighbors')
```

```
[44]: Text(0.5, 0, 'n_neighbors')
```



4 Courbe de validation

La courbe de validation est un outil essentiel dans l'arsenal de l'évaluation des modèles de machine learning. Elle nous permet d'explorer comment les performances d'un modèle varient en fonction d'un hyperparamètre spécifique. Comprendre comment choisir les valeurs optimales pour les hyperparamètres est crucial pour obtenir des modèles performants. Rôle de la Courbe de Validation

La courbe de validation joue un rôle fondamental dans le réglage des hyperparamètres d'un modèle. Elle nous permet de répondre à des questions telles que :

- **Overfitting et Underfitting** : La courbe de validation nous aide à identifier si notre modèle souffre de surajustement (overfitting) ou de sous-ajustement (underfitting). Si un modèle est surajusté, ses performances seront excellentes sur les données d'entraînement mais médiocres sur les données de test. Si un modèle est sous-ajusté, ses performances seront médiocres sur les données d'entraînement et de test.
- **Sélection d'Hyperparamètres** : La courbe de validation nous aide à sélectionner les valeurs optimales pour les hyperparamètres du modèle. Par exemple, dans un modèle K-Nearest Neighbors (KNN), nous pouvons utiliser la courbe de validation pour déterminer la meilleure valeur de k (le nombre de voisins) qui minimise l'erreur de validation.
- **Compréhension du Comportement du Modèle** : Elle nous donne un aperçu du comportement général du modèle en réponse aux changements d'hyperparamètres. Cela peut nous aider à comprendre comment le modèle fonctionne et pourquoi il se comporte de telle ou telle manière.

```
[45]: from sklearn.model_selection import validation_curve
```

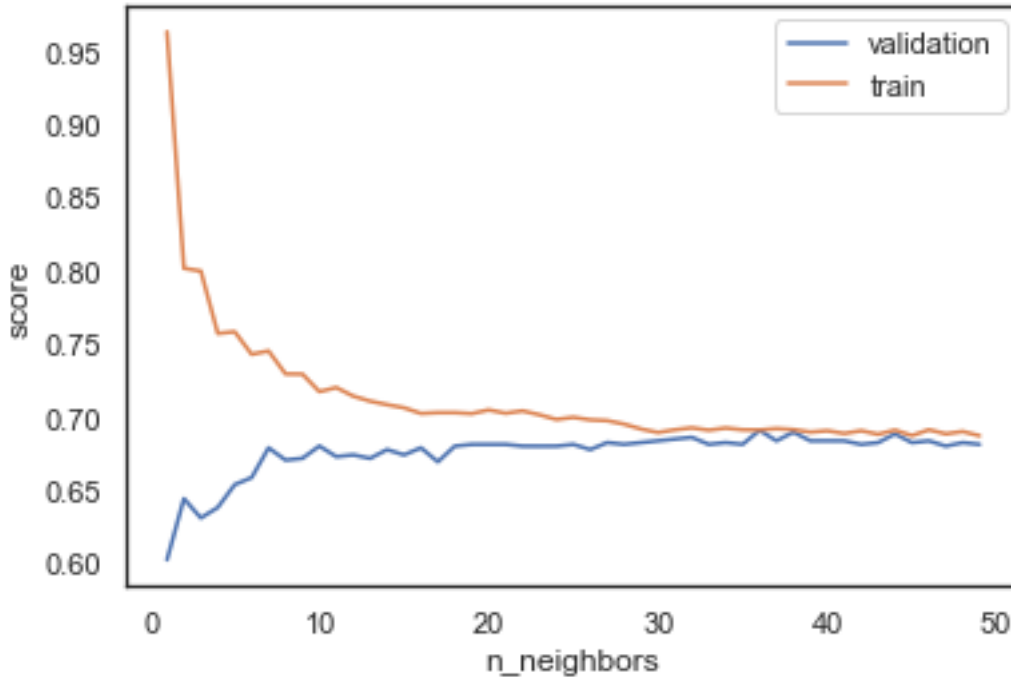
```
[46]: k = np.arange(1, 50)

train_score, val_score = validation_curve(model, X_train, y_train, param_name='n_neighbors', param_range=k, cv=10)

plt.plot(k, val_score.mean(axis=1), label='validation')
plt.plot(k, train_score.mean(axis=1), label='train')

plt.ylabel('score')
plt.xlabel('n_neighbors')
plt.legend()
```

```
[46]: <matplotlib.legend.Legend at 0x7f8d85bb3e50>
```



5 GridSearchCV

Grid Search CV, ou recherche en grille avec validation croisée, est une technique essentielle pour le réglage des hyperparamètres d'un modèle de machine learning. Elle nous permet de systématiquement explorer un ensemble prédéfini de combinaisons d'hyperparamètres et de sélectionner celle qui donne les meilleures performances de modèle. Rôle de Grid Search CV

Le rôle de Grid Search CV est crucial dans le développement de modèles de machine learning pour plusieurs raisons :

- **Optimisation des Hyperparamètres** : Les modèles de machine learning ont souvent des hyperparamètres qui ne peuvent pas être appris à partir des données, comme le taux d'apprentissage, la profondeur d'un arbre de décision, ou le nombre de voisins dans K-Nearest Neighbors. Grid Search CV nous permet d'optimiser ces hyperparamètres en testant différentes combinaisons.
- **Recherche Systématique** : Grid Search CV effectue une recherche systématique dans l'espace des hyperparamètres. Au lieu de deviner quelles valeurs fonctionneront le mieux, Grid Search CV teste chaque combinaison de manière exhaustive, ce qui garantit que nous ne manquons pas les meilleurs réglages.
- **Éviter le Surajustement** : En utilisant la validation croisée, Grid Search CV nous aide à éviter le surajustement en évaluant les performances du modèle sur des ensembles de données de validation indépendants.
- **Comparaison de Modèles** : Grid Search CV permet de comparer plusieurs modèles de manière

équitable en optimisant leurs hyperparamètres respectifs. Cela nous aide à sélectionner le modèle le plus performant pour notre tâche.

```
[47]: from sklearn.model_selection import GridSearchCV
```

Le code suivant implémente le processus de Grid Search CV avec un exemple d'un modèle K-Nearest Neighbors. Le Processus de Grid Search CV se décompose en plusieurs étapes :

1. **Définition de la Grille d'Hyperparamètres (param_grid) :** Vous commencez par définir une grille d'hyperparamètres que vous souhaitez optimiser. Dans cet exemple, vous optimisez deux hyperparamètres pour le modèle KNN : le nombre de voisins (`n_neighbors`) et la métrique de distance (`metric`). Vous spécifiez une plage de valeurs pour `n_neighbors` de 1 à 19 inclus et deux options de métrique de distance : euclidienne et de Manhattan.
2. **Création de l'objet GridSearchCV :** Vous créez un objet `grid` de la classe `GridSearchCV`. Cet objet prend en argument le modèle que vous souhaitez optimiser (dans ce cas, `KNeighborsClassifier()`), la grille d'hyperparamètres (`param_grid`), et le nombre de plis pour la validation croisée (dans cet exemple, `cv=5` signifie une validation croisée à 5 plis).
3. **Exécution de la Recherche en Grille :** En appelant la méthode `fit()` sur l'objet `grid`, vous lancez la recherche en grille. Scikit-Learn va automatiquement effectuer toutes les combinaisons possibles des valeurs d'hyperparamètres que vous avez spécifiées dans `param_grid`. Pour chaque combinaison, il va effectuer une validation croisée (dans ce cas, à 5 plis) et enregistrer les résultats.
4. **Identification de la Meilleure Combinaison :** Une fois la recherche terminée, vous pouvez accéder aux résultats à l'aide des attributs de l'objet `grid`. `grid.best_estimator_` vous donne le modèle avec la meilleure combinaison d'hyperparamètres, `grid.best_score_` vous donne le score moyen sur les ensembles de validation pour cette meilleure combinaison, et `grid.best_params_` vous donne les valeurs spécifiques des hyperparamètres qui ont donné ce meilleur score.
5. **Évaluation du Modèle Optimal :** Enfin, vous pouvez évaluer le modèle optimal sur un ensemble de test indépendant en utilisant `model.score(X_test, y_test)`.

```
[48]: param_grid = {'n_neighbors': np.arange(1, 20),  
                  'metric': ['euclidean', 'manhattan']}  
  
grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5)  
  
grid.fit(X_train, y_train)  
model = grid.best_estimator_  
grid.best_score_, grid.best_params_, model.score(X_test, y_test)
```

```
[48]: (0.6902195608782434,  
      {'metric': 'manhattan', 'n_neighbors': 19},  
      0.645933014354067)
```

6 Learning Curve

La courbe d'apprentissage est un outil essentiel pour évaluer la performance d'un modèle d'apprentissage automatique en fonction de la quantité de données d'entraînement. Elle permet de comprendre comment la taille de l'ensemble de données affecte la capacité du modèle à généraliser et à éviter le surajustement ou le sous-ajustement. Rôle de la Courbe d'Apprentissage

La courbe d'apprentissage répond à deux questions fondamentales lors de l'entraînement d'un modèle :

- Surajustement (Overfitting) : Le modèle est-il trop complexe et a-t-il appris les données d'entraînement au point de mal généraliser sur de nouvelles données (surajustement) ?
- Sous-ajustement (Underfitting) : Le modèle est-il trop simple et ne parvient-il pas à capturer les relations sous-jacentes dans les données (sous-ajustement) ?

La courbe d'apprentissage aide à visualiser ces problèmes potentiels en traçant les performances du modèle (par exemple, l'exactitude ou l'erreur) sur les ensembles d'entraînement et de validation en fonction de la taille de l'ensemble de données d'entraînement. En examinant ces courbes, vous pouvez déterminer si le modèle bénéficierait de plus de données d'entraînement ou d'un modèle plus complexe.

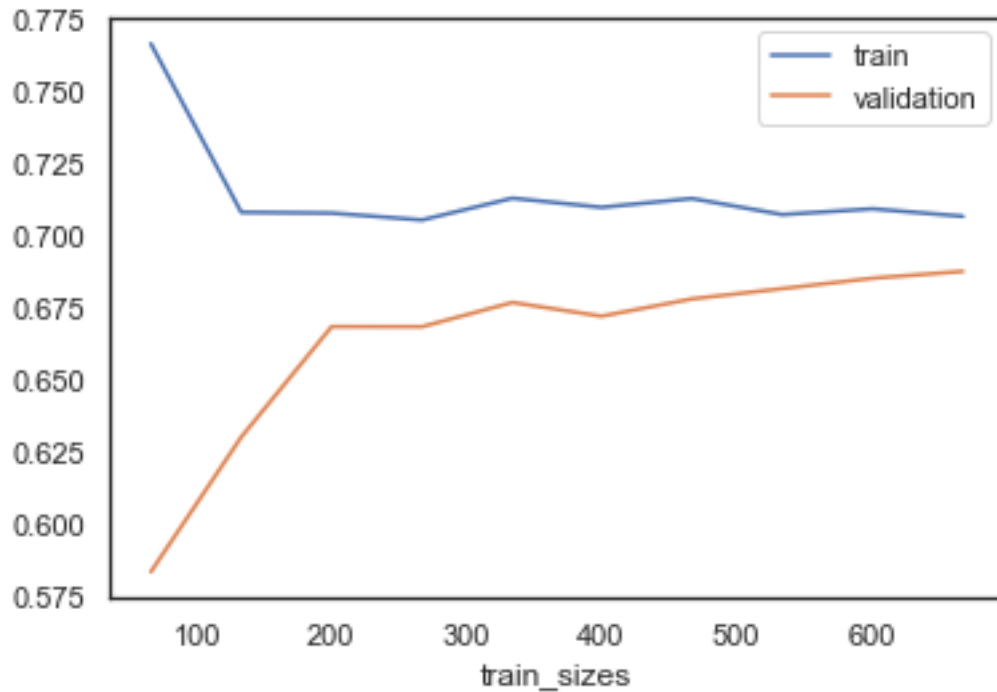
```
[49]: from sklearn.model_selection import learning_curve

[50]: N, train_score, val_score = learning_curve(model, X_train, y_train,
                                              train_sizes=np.linspace(0.1, 1, 10),
                                              cv=5)

print(N)
plt.plot(N, train_score.mean(axis=1), label='train')
plt.plot(N, val_score.mean(axis=1), label='validation')
plt.xlabel('train_sizes')
plt.legend()
```

```
[ 66 133 200 267 334 400 467 534 601 668]
```

```
[50]: <matplotlib.legend.Legend at 0x7f8d85962f40>
```



7 Exercice 1

Pour cet exercice, appliquez ce que nous avons appris sur la sélection de modèles sur le jeu de données Titanic : - Créer un Train-Set et un Test-set. s'entraîner puis évaluer - Avec GridSearch, trouvez les meilleurs hyper-paramètres, métriques et pondérations `n_neighbors` - La collecte de plus de données sera-t-elle utile ?

8 Exercice 2

Contexte : Selon l'Organisation mondiale de la santé (OMS), l'accident vasculaire cérébral (AVC) est la deuxième cause de décès dans le monde, responsable d'environ 11 % de l'ensemble des décès. Ce jeu de données est utilisé pour prédire si un patient est susceptible de faire un AVC en se basant sur les paramètres d'entrée tels que le sexe, l'âge, diverses maladies et le statut tabagique. Chaque ligne dans les données fournit des informations pertinentes sur le patient.

En nous basant sur ce que vous avez appris en DEA, en prétraitement des données et en évaluation des modèles, nous avons les tâches suivantes à exécuter :

1. Effectuer une analyse exploratoire de ces données.
2. Préparer un pipeline de prétraitement pour l'ensemble de données.
3. Entraîner un modèle KNN et régler les hyperparamètres.

Description des attributs du jeu de données : Informations sur les attributs

- id : identifiant unique

- gender : “Homme”, “Femme” ou “Autre”
- age : âge du patient
- hypertension : 0 si le patient n’a pas d’hypertension, 1 si le patient a de l’hypertension
- heart_disease : 0 si le patient n’a pas de maladie cardiaque, 1 si le patient a une maladie cardiaque
- ever_married : “Non” ou “Oui”
- work_type : “enfants”, “Fonctionnaire”, “Jamais travaillé”, “Privé” ou “Indépendant”
- Residence_type : “Rural” ou “Urbain”
- avg_glucose_level : taux de glucose moyen dans le sang
- bmi : indice de masse corporelle
- smoking_status : “fumeur occasionnel”, “non-fumeur”, “fumeur” ou “Inconnu”*
- stroke : 1 si le patient a eu un AVC ou 0 sinon

** Remarque ** : “Inconnu” dans smoking_status signifie que l’information n’est pas disponible pour ce patient. Dans cet exercice, vous serez amenés à explorer, prétraiter et créer un modèle de prédiction des AVC en utilisant les techniques d’analyse exploratoire des données, de prétraitement des données et d’évaluation des modèles que vous avez apprises.